

Usage Metering & Billing Engine

Build the service every SaaS needs: how much has this customer used, what does it cost, and have they hit their limit? Metering, quotas, correct money math, and Stripe test mode — where correctness really matters.

Difficulty: Medium

Self-paced · no deadlines

JavaScript or Python

Public GitHub repo

\$0 · no credit card, ever

THE FLAVOR

Money and limits — the most bounded scope, and the one where bugs cost real money

YOU WILL MASTER

Idempotent metering · quota enforcement · money math · Stripe webhooks

YOUR \$0 STACK

Node or Python · Docker Postgres · Stripe test mode + Stripe CLI (all free)

How to read this document: § 1–2 tell you what this capstone is and whether it's the right pick for you. § 3–9 are the build: rules, features, architecture, the definition of done, and the build phases. § 10–12 are the practical frame: the free tools, the GitHub rules, and exactly how you'll be evaluated. § 13–14 prepare your demo and give you hand-picked, free resources for every phase. You don't need to memorize it — work through the § 8 phases at your own pace and come back when you need detail.

Contents

1. The mission.....	1	9. Stretch goals	6
2. What it takes to finish.....	2	10. Your \$0 stack	6
3. Ground rules	2	11. GitHub rules.....	8
4. What you'll build.....	3	12. How it's evaluated	9
5. Architecture overview	4	13. The final demo	10
6. Definition of done	5	14. Curated resources.....	11
7. Realistic scope	5	15. Glossary	12
8. The build, phase by phase	5		

1 · The mission

Every SaaS product on Earth must answer three questions: **How much has this customer used? How much should they pay? Have they reached their plan limits?** In this capstone you build the backend service that answers all three. You'll meter usage, enforce subscription quotas, calculate costs — including the genuinely tricky AI-token pricing rules — and integrate **Stripe in test mode** for subscription management, with signature-verified, idempotent webhooks keeping plans in sync.

Billing systems look simple from the outside. Then you meet the real world: a network retry that must not double-charge, a webhook that arrives twice, a customer exactly at their quota boundary. **A single bug can mean double-charging customers, giving away unlimited access, or losing revenue.** This capstone is about building those systems safely.

And a career note: billing is where a lot of engineers are quietly terrified — which makes being calmly good at it unusually valuable. "I built a metering and billing engine with proven no-double-count guarantees" is a sentence interviewers remember.

Newer to backend? This is the recommended pick. The most bounded scope on the menu: two plans, two usage types, one dummy billable endpoint. No AI required anywhere in the core. Every hard part is a *correctness* puzzle, not an infrastructure one.

2 · What it takes to finish

Honest picture before you commit — **Medium**, with the difficulty concentrated in precision, not size:

The three genuinely hard parts

- **Exactly-once metering.** The same request retried must record exactly one usage event. Your idempotency-key design — and the test that proves it — is the heart of the capstone.
- **Boundary honesty.** At 999 of 1,000 calls, what happens? At exactly 1,000? Your quota logic and its [429 / 402](#) responses must be exact, tested, and explainable.
- **Token pricing rules.** Cached input tokens are cheaper; reasoning tokens count as output; categories can't just be added together. The math is easy — encoding it *correctly, with pinned tests* is the discipline.

Time budget: roughly **30–45 focused hours**, at your own pace — the leanest capstone build. Spend the saved hours on tests; in billing, tests *are* the product.

You already have the parts

Piece of this capstone	Where you already built it
Idempotent endpoints	A6
Quota API + honest status codes	A5
Cost math + the AI-token gotchas	A12
Stripe-style signed webhooks	A14
Multi-tenant data modeling	A3

Pick this one if you want the clearest path to a genuinely excellent result — or if "correct under retries, failures, and real-world conditions" sounds like the engineering identity you want to build.

3 · Ground rules — read before you start

These rules are the same for every capstone in the track. They exist so that 20,000+ interns can be evaluated fairly — and so your finished project is something you can safely show in an interview.

The five rules

Rule	What it means for you
------	-----------------------

Pick one, early	This internship is self-paced — no deadlines. Still, choose your capstone early (once the first assignments have shown you the landscape) and write a one-page design doc (problem, data model, API surface, layer sketch, one explicit non-goal). Phase 1 in § 8 is exactly that doc.
One separate, public repo	The capstone lives in its own public GitHub repository from day one — never inside your assignments repo. Full rules in § 11.
\$0, no credit card — ever	Everything can be built with free tools; this document lists the exact free stack in § 10. If you ever find yourself on a page asking for a credit card, stop — you took a wrong turn, the free path exists.
AI-assisted building is encouraged — and owned	Use AI tools freely, but keep <code>BUILDLLOG.md</code> honest: where AI helped, where it was wrong, what you changed. At the demo you will be asked to explain 2–3 lines the evaluator picks. <i>"The AI wrote it"</i> is not an answer.
Propose your own	You may pitch a different capstone idea against the shared requirements (§ 12). If it exercises a comparable skill set, mentors can approve it.

CONSTRAINTS FOR THIS CAPSTONE

- **Stripe test mode only.** It's free, needs no card, and moves no real money — test cards like `4242 4242 4242 4242` work with any future expiry. Never switch to live mode; there is no reason to.
- **Stripe secrets stay in `.env`** (git-ignored): API key and the `whsec_` webhook secret. A committed Stripe key — even a test one — is an instant repo-hygiene fail.
- **Store money as integers** (cents / micro-units), never floats. The § 14 reading explains why.
- **Use the Stripe CLI** to forward and replay webhooks locally — no public URL or tunnel needed.

4 · What you'll build

Customers belong to **tenants**; each tenant has a subscription plan with quotas:

Plan	API calls	AI tokens
Free	1,000 / month	100k / month
Pro	Higher limits	Higher limits

Your service handles four concerns:

1 Usage metering

teaches: idempotency

Every billable action records a usage event attributed to the tenant:

```
Tenant A generated an AI response
→ record 2,500 output tokens
→ store usage event
```

The system must be **idempotent**: same request + same idempotency key = **one usage event only**. Retries must never create duplicate charges — this is the bug that overcharges real customers.

2 Quota enforcement

teaches: honest API boundaries

Before allowing a billable action: current usage + requested usage → check plan limits → allow or reject. At the limit, respond honestly and helpfully:

- 429 Too Many Requests → usage quota exceeded
- 402 Payment Required → upgrade/payment required

The API must clearly explain *why* a request was blocked. Status codes are how machines read your answers.

3 Cost calculation

teaches: money math

Convert usage into money — API calls to a monthly cost, and AI tokens with the real-world pricing rules:

```
input tokens + cached input tokens + output tokens + reasoning tokens → total cost
· cached input tokens are cheaper
· reasoning tokens count as output tokens
· token categories cannot simply be added together
```

Pricing constants pinned in config and **covered by tests** — model it on FlyRank's `chat-pricing.config.ts` and its test file.

4 Stripe subscription integration (test mode)

teaches: safe payment integration

A Checkout flow (*customer picks Pro* → *Stripe Checkout* → *subscription created*) and a webhook handler for `checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`. Your backend must: **verify the webhook signature**, **prevent duplicate event processing**, and **update the tenant's plan/status**. Payment truth lives at Stripe; your database mirrors it through verified events only.

5 · Architecture overview

One metering path, one read path, one payment-sync path — small on purpose, correct by construction:

```
Client → Billable API request
    ↳ MeterService.record(tenant, type, qty, idempotencyKey)
        ↳ duplicate key? → return original result (no new event)
        ↳ store usage_event
    ↳ Quota Check → allowed
        ↳ limit exceeded → 402 / 429 + clear message

GET /usage ← rollup(usage_events) → { used, limit, cost }

Stripe Checkout (test mode) → subscription created
Stripe -signed webhook→ /webhooks/stripe
    ↳ verify signature (forged → 400)
    ↳ deduplicate event (replay → ignored)
    ↳ update tenant plan / status
```

6 · Definition of done — the core checklist

This is the contract. **Done = every box below ticked, with one pasted proof per box in `EVIDENCE.md`**. Each box is written so a reviewer can verify it in minutes.

METERING

- ☐ A billable action creates exactly one usage event, even under retries — deduplicated by idempotency key.
- ☐ A test proves double-counting cannot happen.

QUOTAS

- ☐ Usage is checked against the tenant's plan; requests over the limit are rejected.
- ☐ Responses carry the correct status codes (`429` / `402`) and a message explaining why.

COST CALCULATION

- ☐ Monthly usage rolls up into a cost figure per tenant.
- ☐ AI token pricing handles cached input tokens, reasoning tokens, and output pricing correctly.
- ☐ Pricing constants are pinned and covered by tests.

STRIPE INTEGRATION

- ☐ Subscription checkout works end-to-end in Stripe test mode.
- ☐ Webhooks verify signatures, ignore duplicate events, and update tenant plan/status.

DATA MODEL, TESTS & DOCUMENTATION

- ☐ Database includes tenants, plans, subscriptions, and usage events; customer data isolated per tenant.
- ☐ Tests cover: duplicate usage prevention, quota boundary cases (at / just under / over), cost calculations, invalid-webhook rejection, duplicate-webhook handling.
- ☐ README + architecture diagram + setup instructions; submission-pack files from § 11 present.

7 · Realistic scope — where to stop

You do **not** need real payments — Stripe test mode is exactly right. Keep the system intentionally small:

- **2 plans** (Free / Pro) · **2 usage types** (API calls + AI tokens) · **1 dummy billable endpoint** (e.g. `POST /generate` → creates usage event → checks quota → calculates cost). That exercises every rule.
- **No invoicing, proration, or overage billing in core** — those are stretch goals with real teeth.
- **The AI tokens can be simulated**. You're metering numbers, not calling a model — no AI key needed at all.
- **Use the Stripe CLI** (`stripe listen`, `stripe trigger`) to replay webhook events locally.

8 · The build, phase by phase

This internship is **self-paced** — there is no calendar and no deadline. Work through the phases **in order, at your own speed**: the track assignments are the parts, the capstone assembles them. Each phase ends with a **gate** — a concrete result that tells you it's safe to move on. The effort estimates are just orientation; take what you need. Short on time overall? Shrink scope (§ 7), don't skip phases.

Phase 1 Design · ≈4–6 h

gate: design doc signed off

- Database schema: tenants, plans, subscriptions, usage events
- Plans + quotas defined
- The metering API contract and idempotency strategy

Phase 2 Core billing logic · ≈9–13 h

gate: the double-count test passes; boundary returns 429/402

- Idempotent usage tracking with duplicate prevention
- Quota enforcement with honest status codes

Phase 3 Stripe integration · ≈8–12 h

gate: test Checkout flips a tenant Free → Pro via webhook

- Checkout flow in test mode
- Webhook verification + deduplication
- Subscription/plan synchronization

Phase 4 Cost & finalization · ≈7–10 h

gate: /usage numbers match your pinned tests; all tests green

- Cost rollups with the AI-token rules
- Full test suite · README + diagram · **EVIDENCE.md** filled as you go

Phase 5 Demo prep · ≈2–3 h

gate: \$ 13 rehearsed twice, forged-webhook moment ready

- Seed a tenant near its quota; rehearse the boundary, the retry, and the upgrade

9 · Stretch goals — only if the core ships

★ **Optional, and only after every § 6 box is green.** A finished core with one polished stretch beats three half-stretches. Each of these is a genuine "I went deep" interview story:

- **Overage billing:** allow usage beyond limits and calculate the additional charges (+ projected cost).
- **Invoices:** monthly statements with usage line items.
- **Usage alerts:** notify customers at 80% and 100% of quota.
- **Proration:** handle a mid-cycle upgrade correctly — genuinely tricky, a great "I went deep" story.
- **Reconciliation job:** a nightly comparison of your database against Stripe's view — catches missed webhooks.

10 · Your \$0 stack — the free-tools promise

We promised you can finish this internship without paying for anything. Here is the proof for this capstone. Every requirement maps to a tool that is free with **no credit card**:

You need	Free tool (no credit card)	Notes
Language + framework	Node.js + Express or Python + FastAPI	Free, as all track long

Database	PostgreSQL via Docker (or SQLite)	Free
Payments	Stripe test mode	Free · no card · test card 4242... · no real money ever
Local webhook delivery	Stripe CLI (<code>stripe listen --forward-to localhost</code>)	Free · replays events with <code>stripe trigger</code>
AI usage to meter	Simulated token counts (no model call needed)	Free · metering numbers, not AI
Repo	GitHub (public)	Free

The iron rule: if any tool, tier, or tutorial asks for a credit card, it is the wrong path — a free alternative for this capstone exists in the table above. Stuck anyway? Ask in the community before paying for *anything*.

11 · GitHub rules — your public repo

Your capstone is also your **portfolio piece**. These rules make the repo something a recruiter, a mentor, and our automated evaluator can all navigate without asking you anything.

The non-negotiables

- ☐ **One dedicated repository, public from day one.** Create it the day you choose your capstone. Do **not** build in your assignments repo, a private repo you flip later, or a monorepo shared with other work. Progress in the open is part of the story — an honest half-built history beats a single "final version" push.
- ☐ **Name it clearly:** `flyrank-capstone-<short-name>` (for this capstone we suggest `flyrank-capstone-metering-billing`). Lowercase, hyphens, no spaces.
- ☐ **Commit as you build.** Small, meaningful commits with messages that say what changed (`Add idempotency key check to publish endpoint`, not `update stuff`). Aim for at least one commit per working session; each phase in § 8 should be visible in your history.
- ☐ **Never commit a secret.** No API keys, tokens, passwords, or `.env` files — put `.env` in `.gitignore` before your first commit and ship a `.env.example` with placeholder values instead. A leaked key in your history means rotating the key and rewriting history — ask for help the moment it happens.
- ☐ **A stranger can run it.** The README's setup section must work on a clean machine with one documented run command (`docker compose up` or equivalent) plus a seed step for demo data.

Required files at submission

File	What goes in it
<code>README.md</code>	What the system does, an architecture diagram (an image or ASCII sketch), exact run + seed steps, and an honest "limitations" note.
<code>capstone.yaml</code>	A small manifest the evaluator reads: <code>run:</code> (one command), <code>seed:</code> , <code>test:</code> , <code>base_url:</code> and the endpoints to probe.
<code>EVIDENCE.md</code>	One pasted proof per Definition-of-Done checkbox in § 6 — a test name + output, a curl transcript, or a log line. Claims without evidence score as not done.
<code>BUILDLOG.md</code>	Your AI-usage log: where AI helped, where it was wrong, what you changed. Honesty is graded, perfection is not.
<code>.env.example</code>	Every environment variable the app needs, with safe placeholder values.

DO

- Create the repo public before you build, first commit = README skeleton + `.gitignore`.
- Add a license (MIT is a fine default) so others may read and learn from it.
- Use branches if you like — but keep `main` always runnable.
- Paste real command output into `EVIDENCE.md` as you finish each checkbox — not in a panic at the end.

DON'T

- Don't mix capstone code with track assignments — separate repos, always.
- Don't commit `node_modules/`, `virtualenvs`, or datasets over a few MB.
- Don't force-push over your history before submission — the journey is evidence.
- Don't commit real tokens "just for a second". There is no such thing on a public repo.

12 · How your capstone is evaluated

Three layers, published up front — **you know exactly what will be checked, so build to pass it.**

Layer 1 — The submission pack (machine-checkable)

The portal first checks your repo structure: the five required files from § 11, a `run:` command that boots the system, and a `test:` command that runs green. Missing pack files cost points before a human ever looks.

Layer 2 — Acceptance probes (behavioral, pass/fail)

An evaluator (human or automated) runs these against your live system. They are not secrets — they are promises:

PROBE 1 — Send the same billable request twice with one idempotency key → exactly one usage event; the second response mirrors the first.

PROBE 2 — Drive a tenant to its exact quota → the request at the boundary behaves per your documented rule; the one after returns `429 / 402` with a clear message.

PROBE 3 — Complete a Stripe test Checkout → the webhook flips the tenant Free → Pro; `GET /usage` shows the new limits.

PROBE 4 — Send a forged webhook (bad signature) → `400`, nothing changes. Replay a real event twice → processed once.

PROBE 5 — Run the pinned pricing tests → cached-input and reasoning-token rules produce the exact expected totals; `GET /usage` matches.

Layer 3 — The rubric (judgment, 1–5 per dimension)

The weights say what we value: **correctness and resilience over feature count.**

Dimension	What a "5" looks like	Weight
Architecture	Clean layers; swap the DB or a provider without touching business logic	×3
Correctness	Happy path solid; edge cases handled; probes all pass	×3
Resilience	Survives bad input, a dead dependency, a job running twice	×3
Security	Real authorization; validated input; secrets encrypted & never logged	×2
AI cost & grounding (if AI used)	Cost metered + capped; outputs grounded; eval catches regressions	×2
Testing	The scary cases, deterministically; evals for AI behavior	×2
Communication	README + diagram tell the story; honest about limits; demo lands	×2

Levels: **Ships** = every core box in § 6 done + all probes pass (a real pass) · **Solid** = Ships + rubric average ≥ 3.5 · **Exceptional** = Solid + meaningful stretch goals. A small system that is correct, resilient and well-tested beats a huge one that falls over — that is what senior engineers actually value.

The shared requirements (every capstone must show these)

These eight patterns are how we know you absorbed the program, not just this topic. You built each one in an assignment already:

#	Requirement	Where you learned it
---	-------------	----------------------

1	Layered architecture — data / logic / HTTP separated	A2 + the architecture live event
2	Validation at the boundary — bad input → clean 4xx, never a 500	A5
3	≥1 background job — slow/bulk work off the request path, retries + failure alert	A7
4	Real persistence — schema as migrations, right indexes, isolated tenants	A5
5	Idempotency where it matters — the retried action happens once	A5 stretch + A10
6	Secrets clean — env only, encrypted if stored, never logged	A10
7	Cost tracked, if AI is used — per call, attributed, with a budget guard	A6
8	Tests that matter — the scary cases, deterministic; AI features get an eval	A13

13 · The final demo — your 6 minutes

Rehearse this flow start to finish. A demo that shows **one failure handled gracefully** impresses more than ten happy paths.

1. Make billable API calls until the tenant hits its quota → show the clean refusal at the boundary ([429](#) , explained).
2. Retry the same request with the same idempotency key → **usage did not double-count**. Show the single event row.
3. Run a Stripe test-mode Checkout → the webhook fires → the tenant flips from Free to Pro live.
4. Send a forged webhook → rejected with [400](#) . Replay the real one → ignored as a duplicate.
5. Finish on [GET /usage](#) : used / limit / cost adding up *exactly*, with your pinned cost tests green on screen.
6. Closing line: *"usage, money, and customer access stay correct under retries, failures, and real-world conditions."*

Study these FlyRank parts first

FlyRank reference	What to steal from it
config/chat-pricing.config.ts + tests/chat-pricing.config.test.ts	Cost math done right: token categories, pricing constants, pinned calculation tests
app/api/webhooks/stripe/route.ts	The production Stripe webhook: signature verification + plan/status sync
Patterns across inngest/	Retries, duplicate prevention, reliable processing — the idempotency mindset

14 · Curated resources — free, verified, leveled

Don't read everything. Each row says when to reach for it. Every link was verified while writing this brief; every resource is free with no credit card. If a link ever dies, the title is searchable.

● **Start here** (everyone) · ● **When you're comfortable** · ● **Optional deep dive** | Lanes: **JS** · **PY** · **both**

Phase 1 · Design

Lane	Resource	Format	When to use it
● both	Stripe — Designing APIs with idempotency	Article, ~7 min	Read before designing the metering endpoint — why retries need keys.
● both	Stripe — Usage metering: a guide	Article, ~6 min	Vocabulary check before schema design: collection → aggregation → billing.
● both	Modern Treasury — Floats don't work for storing cents	Article, ~10 min	Before choosing money columns — the case for integer cents.

Phase 2 · Metering & quotas

Lane	Resource	Format	When to use it
● both	Stripe API — Idempotent requests	Docs, ~5 min	A reference implementation to model your own Idempotency-Key handling on.
● both	MDN — 429 Too Many Requests	Reference, ~3 min	When wiring quota-exceeded responses (include Retry-After).
● both	MDN — 402 Payment Required	Reference, ~3 min	When deciding 402 vs 429 semantics for lapsed/unpaid plans.

Phase 3 · Stripe integration

Lane	Resource	Format	When to use it
● both	Stripe — Test mode & sandboxes	Docs, ~10 min	First stop: free test cards, no real money, no credit card needed.
● both	Stripe — Billing quickstart (subscriptions)	Docs + code, ~45 min	The canonical Checkout walkthrough — toggle Node or Python samples.
● both	Stripe — Receive webhook events	Docs, ~20 min	Core reading for the handler: stripe listen , retries, event ordering.
● both	Stripe — Verify webhook signatures	Docs, ~10 min	When verification fails: raw-body pitfalls, whsec_ secrets.
● both	Stripe CLI — Get started	Docs, ~10 min	Install before local webhook testing.
● both	Stripe CLI — stripe trigger	Reference, ~5 min	Replay checkout.session.completed & friends without clicking through Checkout.
● JS	Stripe subscriptions + webhooks with Node.js	Video, ~50 min	Express-lane build-along: Checkout → verified webhook → subscription state.

●	PY	TestDriven.io — Flask Stripe subscriptions	Tutorial, ~1 h	Python-lane build-along (Flask patterns port directly to FastAPI).
---	----	--	----------------	--

Phase 4 · Cost & hardening

Lane	Resource	Format	When to use it
● both	Gemini API pricing (cached input + thinking tokens)	Reference, ~5 min	Ground truth that token categories price differently — the rules your calculator must encode.

15 · Glossary

Plain-language definitions of the **bold** terms in this brief. No definition depends on another — read in any order.

Term	What it means
Tenant	One customer organization in a multi-tenant system. Every usage event, plan, and subscription belongs to exactly one tenant, and tenants never see each other's data.
Usage event	One recorded row of billable activity: tenant, type (API call / tokens), quantity, timestamp, idempotency key.
Idempotency key	A unique value sent with a request so a retry can be recognized as "already done" — the mechanism that prevents double-counting.
Quota	A plan's monthly allowance (1,000 API calls, 100k tokens). Enforced <i>before</i> the action, not after.
402 Payment Required	The status code for "your plan doesn't allow this — upgrade or pay". Distinct from 429.
429 Too Many Requests	The status code for "you've exceeded your usage limit / rate". Pair it with a clear message.
Rollup	Aggregating many usage events into one summary: used, limit, cost for the month.
Cached input tokens	Input tokens the AI provider already had cached — billed cheaper than fresh input. Your calculator must price them separately.
Reasoning tokens	Hidden "thinking" tokens some models produce — billed as output tokens, not a separate free category.
Stripe test mode	Stripe's free sandbox: test cards, real API shapes, zero real money. Everything this capstone needs.
Checkout	Stripe's hosted payment page — your backend creates a session, the customer "pays" with a test card, a webhook tells you the result.
Webhook signature	The cryptographic stamp proving an event really came from Stripe. Verify first; forgeries get 400 .
Stripe CLI	The free command-line tool that forwards Stripe webhooks to localhost and replays events (stripe trigger).
Proration	Charging a fair partial amount when a plan changes mid-billing-cycle — a stretch goal with real teeth.

FlyRank Internship · Backend Development Track · Capstone — Usage Metering & Billing Engine. Everything in this brief can be completed with free tools; no resource linked here requires a credit card. Questions → the capstone channel on the community. Version: July 2026.